

SIEM Threat Detection Dashboard

S. Md. Afzal

github.com/SYEDMAHAMMEDAFZAL/siem-dashboard | linkedin.com/in/syed-mahammed-afzal | 2026

Abstract — This report documents the design and implementation of a custom Python-based SIEM (Security Information and Event Management) Threat Detection Dashboard. The system simulates real-world attack scenarios, detects threats in real time using rule-based analysis, classifies alerts by severity, and displays findings on a live Flask web dashboard — built entirely from scratch without any commercial SIEM tool.

Keywords: SIEM, Threat Detection, Brute Force, Port Scan, Malware, Python, Flask, Log Analysis, SOC, Cybersecurity Dashboard

1. WHAT IS SIEM?

SIEM stands for Security Information and Event Management. It is a core technology used by Security Operations Centre (SOC) teams worldwide. A SIEM system collects logs from all devices on a network, analyses them in real time, detects suspicious patterns, and raises alerts for the security team to investigate. Every major cybersecurity job description lists SIEM experience as a mandatory requirement.

Event Type	What It Means	Real World Example
Brute Force Attack	Repeated login attempts from one IP	Hacker trying 1000 passwords on SSH
Port Scan	IP scanning multiple ports rapidly	Attacker doing reconnaissance
Malware Detected	Malicious signature found in traffic	Ransomware / trojan attempting entry
Failed Login	Single or few failed login attempts	Wrong password or credential stuffing
Normal Traffic	Legitimate user activity	Employee browsing internal website

Table 1: Event types detected by the SIEM dashboard.

2. TOOL OVERVIEW

2.1 What the Tool Does

1. log_generator.py simulates 100 real-world attack events across 5 categories.
2. detector.py reads all logs and runs threat detection algorithms.
3. Brute force, port scan, malware and failed login patterns are identified.
4. Each alert is classified as CRITICAL, HIGH, MEDIUM or LOW severity.
5. app.py launches a Flask web server serving a live threat dashboard.
6. Dashboard displays total events, alert counts, active alerts table and top attacker IPs.

2.2 Technology Stack

Component	Technology	Purpose
Language	Python 3	Core scripting and automation
Web Framework	Flask	Live dashboard web server
Data Handling	Pandas	Log data processing and analysis
Log Format	JSON	Structured event storage
Frontend	HTML + CSS	Dashboard UI rendered in browser
OS	Kali Linux	Development and testing environment

Table 2: Technology stack used in this project.

3. SYSTEM DESIGN

3.1 File Structure

```
siem-dashboard/ ■■■ log_generator.py # Simulates attack logs ■■■ detector.py # Threat detection
engine ■■■ app.py # Flask dashboard server ■■■ requirements.txt # Python dependencies ■■■
README.md # Setup and usage guide ■■■ logs/ # Generated JSON log files
```

3.2 Detection Logic

Threat	Detection Rule	Severity
Brute Force	Same IP with 10+ login attempts	HIGH
Port Scan	Single IP scanning 3+ ports	MEDIUM
Repeated Failures	Same IP with 3+ failed logins	LOW
Malware	Malicious signature event in log	CRITICAL

Table 3: Threat detection rules and severity classification.

3.3 Severity Classification

Severity	Meaning	Recommended Action
CRITICAL	Immediate danger — act now	Isolate system. Run full scan.
HIGH	Active attack in progress	Block IP. Enable lockout policy.
MEDIUM	Suspicious reconnaissance	Monitor IP. Check firewall rules.
LOW	Minor suspicious activity	Log and monitor. Rate limit.

Table 4: Alert severity levels and response actions.

4. RESULTS

The system was tested on Kali Linux generating 200 simulated log events. The detector identified 95 alerts across all severity levels. The Flask dashboard rendered all findings in real time at <http://127.0.0.1:5000>.

Metric	Value
Total Events	200
Total Alerts	95
CRITICAL	10 — Malware detections
HIGH	30 — Brute force attacks
MEDIUM	20 — Port scans
LOW	35 — Repeated failed logins
Top Attacker	192.168.1.105 — 15 events

Table 5: Test run results on Kali Linux localhost.

5. HOW TO USE THIS TOOL

5.1 Requirements

- Kali Linux or any Linux distribution.
- Python 3.6 or above.
- pip3 install flask pandas --break-system-packages

5.2 Installation

```
git clone https://github.com/SYEDMAHAMMEDAFZAL/siem-dashboard.git cd siem-dashboard pip3 install
-r requirements.txt --break-system-packages
```

5.3 Run

```
# Step 1 - Generate logs python3 log_generator.py # Step 2 - Start dashboard python3 app.py # Step 3 - Open browser http://127.0.0.1:5000
```

6. SKILLS DEMONSTRATED

Skill	Demonstrated By
SIEM Concepts	Built detection engine from scratch without commercial tools
Python Programming	Three modular scripts with clean separation of concerns
Web Development	Flask server with dynamic HTML dashboard rendering
Threat Detection Logic	Rule-based alert engine with threshold analysis
Log Analysis	JSON log parsing, event correlation, IP tracking
Security Documentation	Professional PDF report with findings and recommendations

Table 6: Technical skills demonstrated through this project.

7. CONCLUSION

This project demonstrates a fully functional, custom-built SIEM threat detection system implemented in pure Python. By building the log generator, detection engine, and dashboard from scratch, the project showcases a deep understanding of SOC workflows, threat detection methodology, and web-based security tooling — skills directly required for SOC Analyst L2 and Security Engineer roles at organisations including IBM Security, Deloitte Cyber, EY Cybersecurity, and equivalent employers.

This is Project 2 of a structured six-month cybersecurity portfolio programme targeting a professional role at 14+ LPA by December 2026.

Legal Notice: This tool is built for educational purposes only. All logs are simulated. Do not use against systems without explicit written permission. The author bears no responsibility for any misuse.

